121 TESTCASES
PROJECT CAB-SYSTEM
Mỗi level gồm 10 test cases, bao phủ: API, business logic, transaction, AI services, event-driven, resilience, security, deployment và monitor.

Test case ID	Test level / Test priority	Test-cases	Context	Input	Expected Result	Student's CHECK ✓	Supervisor's CHECK ✓
0	Điểm danh	Chống điểm ZERO	Điểm danh có mặt báo cáo	Thành viên	Ít nhất một thành viên có mặt		

1	LEVEL 1 – CƠ BẢN (Basic API & Flow)
	
	Mục tiêu: kiểm tra chức năng cơ bản. Hệ thống chạy đúng (happy path)
	
	CI pipeline (unit + integration) Docker hoặc non-docker
	
	Các dịch vụ được test: Auth, Booking, Driver, AI ETA, Pricing, Notification
		Đăng ký user thành công	Hệ thống hoạt động bình thường, chưa tồn tại user	{
				"email": "user@test.com",
				"password": "123456",
				"name": "Test User"
				}
					HTTP 201 Created
					User được lưu DB
					Trả về user_id
					Không có lỗi validation		

2		Đăng nhập trả JWT hợp lệ	User đã tồn tại trong hệ thống	{
				"email": "user@test.com",
				"password": "123456"
				}
					HTTP 200 OK
					Trả về access_token (JWT)
					Token decode hợp lệ (exp, sub):đọc được phần payload (dữ liệu bên trong).
					
					sub (Subject): Đại diện cho user_id hoặc identity. Cho biết token này thuộc về ai.
					
					exp (Expiration Time)
					Thời điểm token hết hạn
					Là timestamp (UNIX time)		

3		Tạo booking với input hợp lệ	User đã login, driver available	{
				"pickup": {"lat": 10.76, "lng": 106.66},
				"drop": {"lat": 10.77, "lng": 106.70},
				"distance_km": 5
				}
					HTTP 200 hoặc 201
					status = REQUESTED hoặc CONFIRMED
					Có booking_id
					Gọi ETA + Pricing thành công		

4		Lấy danh sách booking của user	User đã có ít nhất 1 booking	GET /bookings?user_id=123	HTTP 200 Trả về list booking Mỗi item có booking_id, status		

5		Driver chuyển trạng thái online	Driver tồn tại trong hệ thống	{
				"driver_id": "DRV001",
				"status": "ONLINE"
				}
					HTTP 200
					Driver status updated = ONLINE
					Có thể nhận booking		

6		Booking được tạo với status = REQUESTED	Tạo booking thành công	{
				"pickup": {"lat": 10.76, "lng": 106.66},
				"drop": {"lat": 10.77, "lng": 106.70},
				"distance_km": 5
				}
					status ban đầu = REQUESTED
					Không bị skip sang trạng thái khác
					Có timestamp created_at		

7		Gọi API ETA trả về giá trị > 0	AI ETA service hoạt động	{
				"distance_km": 5,
				"traffic_level": 0.5
				}
					HTTP 200
					eta > 0
					eta hợp lý (ví dụ < 60 phút)		

8		Pricing API trả về giá hợp lệ	Pricing service hoạt động	{
				"distance_km": 5,
				"demand_index": 1.0
				}
					HTTP 200
					price > base fare
					surge >= 1		

9		Notification gửi thành công	Booking đã được tạo	{
				"user_id": "USR123",
				"message": "Your ride is confirmed"
				}
					HTTP 200
					Notification được gửi (log hoặc queue)
					Không lỗi timeout		

10		Logout invalidate token	User đang login	Header: Authorization Bearer token	HTTP 200 Token bị invalidate Gọi lại API với token cũ → 401		

11	LEVEL 2 – VALIDATION & EDGE CASES
	
	Mục tiêu: Hệ thống phản ứng thế nào khi input sai hoặc bất thường?
	
	CI pipeline (unit + integration) Docker hoặc non-docker
	
	Các mục kiểm thử: Validation, Edge cases, Security basic, Idempotency, Error handling.
	
	Không tin input từ client. Hệ thống luôn fail an toàn (fail-safe). Không có hành vi “undefined”
		Booking thiếu pickup → lỗi 400	User gửi request tạo booking nhưng thiếu field bắt buộc	{
				"drop": {"lat": 10.77, "lng": 106.70}
				}
					HTTP 400 Bad Request
					Message: "pickup is required"
					Không tạo booking		

12		Sai format lat/lng → reject	Input sai kiểu dữ liệu (string thay vì float)	{
				"pickup": {"lat": "abc", "lng": 106.66},
				"drop": {"lat": 10.77, "lng": 106.70}
				}
					HTTP 422 Unprocessable Entity
					Validation error từ schema
					Không gọi AI services		

13		Driver offline không nhận booking	Không có driver online trong khu vực	Booking hợp lệ	Booking status = PENDING hoặc FAILED Không assign driver Trả message: "No drivers available"		

14		Payment method invalid → reject	User chọn phương thức thanh toán không tồn tại	{
				"payment_method": "invalid_card"
				}
					HTTP 400
					Message: "Invalid payment method"
					Không gọi Payment Service		

15		ETA với distance = 0	Pickup = Drop	{
				"distance_km": 0
				}
					eta = 0 hoặc rất nhỏ
					Không crash
					Không trả giá trị âm		

16		Pricing với demand_index = 0	Không có nhu cầu (off-peak)
			“Surge” (hay surge pricing) là cơ chế tăng giá cước theo thời gian thực khi nhu cầu (demand) cao hơn nguồn cung (supply) trong hệ thống đặt xe.
			
			surge = max(1.0, demand_index / supply_index)
			
			Nguyên tắc:
			1. Giá KHÔNG BAO GIỜ = 0
			2. Surge KHÔNG BAO GIỜ < 1
			
			demand_index = chỉ số thể hiện mức độ nhu cầu đặt xe
			> 1Nhu cầu cao (giờ cao điểm)
			= 1Bình thường
			< 1Nhu cầu thấp
			= 0Không có nhu cầu
			
			supply_index = chỉ số thể hiện nguồn cung tài xế (drivers available) trong một khu vực tại một thời điểm.
			> 1Nhiều tài xế (dư cung)
			= 1Bình thường
			< 1Ít tài xế (thiếu cung)
				{
				"distance_km": 5,
				"demand_index": 0,
				"supply_index": 1
				}
					surge_multiplier ≥ 1 (không được < 1)
					Giá vẫn hợp lệ
					Không chia cho 0		

17		Fraud API với input thiếu field	Gọi Fraud detection thiếu dữ liệu
			
			Fraud Detection cần gì?
			Hệ thống phát hiện gian lận thường cần nhiều tín hiệu:
			
			user_id
			driver_id
			booking_id
			amount (số tiền)
			location / route
			device_fingerprint
			payment_method
				{
				"user_id": "USR123"
				}
					HTTP 400
					Message: "missing required fields"
					Không chạy model		

18		Token expired → 401	User dùng token hết hạn	Header: Authorization Bearer expired_token	HTTP 401 Unauthorized Message: "Token expired" Không xử lý request		

19		Duplicate booking request (idempotency)	User gửi request giống nhau 2 lần “Duplicate booking (idempotency)” là cơ chế giúp hệ thống không tạo nhiều booking giống nhau khi client gửi lại cùng một request nhiều lần (do retry, network lỗi, double click…).	giống nhau + same idempotency_key	Chỉ tạo 1 booking Request thứ 2 trả kết quả cũ Không tạo duplicate		
20		Input quá lớn (payload size test)	Client gửi request quá lớn	JSON > limit (ví dụ >1MB)	HTTP 413 Payload Too Large Request bị reject Không xử lý logic bên trong		

21	LEVEL 3 – INTEGRATION TEST
	
	Mục tiêu: kiểm tra các service tương tác với nhau đúng cách
	
	CI pipeline (unit + integration)
	
	Các mục kiểm thử:
	Service-to-service,
	AI integration,
	Kafka event,
	API Gateway,
	Retry logic
		Booking → gọi ETA service thành công	Booking Service gọi AI ETA service	{
				"pickup": {"lat": 10.76, "lng": 106.66},
				"drop": {"lat": 10.77, "lng": 106.70}
				}
					Booking API gọi ETA service thành công
					Response chứa eta > 0
					Không timeout		

22		Booking → gọi Pricing service	Booking cần tính giá	{
				"pickup": {"lat": 10.76, "lng": 106.66},
				"drop": {"lat": 10.77, "lng": 106.70}
				}
					Gọi Pricing service
					Trả price > 0
					surge ≥ 1		

23		AI Agent chọn driver từ Driver Service	AI Agent cần chọn driver	danh sách driver từ service	AI Agent nhận danh sách driver Chọn driver hợp lệ Driver tồn tại và online		
24		Booking → Payment → Notification flow	Booking hoàn chỉnh	request booking + payment method	Booking tạo thành công Payment được khởi tạo Notification gửi đến user Flow end-to-end thành công		

25		Kafka event ride_requested được publish	Booking mới được tạo	Bước 1: User đặt xe
				Gọi API /booking
				Bước 2: Booking Service tạo booking
				{
				"booking_id": "BK123",
				"status": "REQUESTED"
				}
				Bước 3: Publish event lên Kafka
				{
				"event_type": "ride_requested",
				"ride_id": "BK123",
				"user_id": "USR001",
				"pickup": {"lat": 10.76, "lng": 106.66},
				"timestamp": "2026-04-04T07:00:00Z"
				}
				Bước 4: Các service khác nhận event
				Matching Service: tìm driver,
				Notification Service: báo user,
				AI Agent: tính ETA, pricing,
				Analytics: lưu log
					Event ride_requested được publish lên Kafka
					Topic đúng; Topic: ride_events
					Payload đúng format		

26		Driver nhận notification	Booking đã assign driver	driver_id	Notification Service gửi message Driver nhận được thông báo Không bị delay lớn		

27		Booking update trạng thái ACCEPTED	Driver accept chuyến	{
				"booking_id": "BK123",
				"status": "ACCEPTED"
				}
					Status chuyển từ REQUESTED → ACCEPTED
					DB update thành công
					Event ride_accepted được publish		

28		MCP context được fetch thành công	AI Agent cần context
			
			Để đưa ra quyết định đúng (chọn tài xế, tính giá, ETA…), AI Agent phải có đủ thông tin (ngữ cảnh) từ nhiều nguồn trong hệ thống.
			
			Context = tập hợp dữ liệu liên quan đến một tình huống hiện tại. Trong Cab Booking: Context = dữ liệu về chuyến đi + tài xế + môi trường + hệ thống
			
			Ví dụ context khi user đặt xe
			{
			"ride_id": "BK123",
			"pickup": {"lat": 10.76, "lng": 106.66},
			"drop": {"lat": 10.77, "lng": 106.70},
			"available_drivers": [
			{"id": "D1", "distance": 2, "rating": 4.8},
			{"id": "D2", "distance": 1, "rating": 4.5}
			],
			"traffic_level": 0.7,
			"demand_index": 1.5,
			"supply_index": 0.8
			}
				request từ AI Agent	MCP Gateway trả context hợp lệ
					Có đầy đủ data (ETA, pricing, driver)
					Không lỗi permission		

29		API Gateway route đúng service	Request đi qua API Gateway	/booking endpoint	Gateway route đúng Booking Service Không route sai service Response đúng		
30		Retry khi Pricing service timeout	Pricing service chậm	simulate timeout	Booking Service retry Hoặc fallback giá Không crash		
31	LEVEL 4 - TRANSACTION Mục tiêu: Hệ thống có giữ được tính nhất quán dữ liệu không Transaction test đảm bảo: Không mất tiền, Không duplicate booking, Không trạng thái lỗi Điểm khó trong microservices: Không có transaction global Phải dùng: Saga pattern, Event-	Transaction tạo booking thành công	User tạo booking hợp lệ	request /booking	Booking được tạo Status = REQUESTED DB commit thành công Không partial write		
32		Rollback khi lỗi giữa chừng	Lỗi xảy ra sau khi insert DB	simulate lỗi sau bước tạo booking	Transaction rollback Không có booking trong DB System consistent		
33		Payment thất bại → rollback booking	Payment service fail	booking + payment	Booking status = FAILED hoặc CANCELLED Không charge tiền Không để trạng thái “dang dở”		
34		Idempotent transaction (duplicate request)	Client retry request	cùng Idempotency-Key	Chỉ 1 transaction được thực hiện Không double charge Trả kết quả cũ		

35	driven, Compensation	Concurrent booking (race condition)	2 request cùng lúc	2 request booking song song	Không tạo duplicate Lock hoặc conflict resolve System consistent		
36		Saga transaction – success flow	Distributed transaction (Booking → Payment → Notification)	full flow	Tất cả bước thành công Saga complete State nhất quán		
37		Saga transaction – failure + compensation	Payment fail sau booking	simulate failure	Trigger compensation Booking → CANCELLED Refund (nếu cần)		
38		Kafka event consistency (outbox pattern)	DB update + publish event	create booking	DB commit và Kafka event đồng bộ Không mất event Không duplicate event		
39		Partial failure (network issue)	Network fail khi gọi Payment	simulate timeout	Retry hoặc fallback Không inconsistent state Transaction không “kẹt”		
40		Data integrity (ACID)	Transaction phức tạp tương ứng nhiều bước DB operation	User đặt xe → tạo booking → tính giá → thanh toán → assign driver	Atomic: tất cả hoặc không gì cả Test: Simulate payment fail Expected: booking bị huỷ DB không có record “dang dở” Consistent: data hợp lệ. Sau transaction, dữ liệu luôn đúng rule business Test: Insert data sai → bị reject Expected: Không commit vào DB Isolated: không bị ảnh hưởng bởi request khác. Transaction không bị “ảnh hưởng” bởi transaction khác. Test: 2 request song song Expected: chỉ 1 thành công không duplicate Durable: commit không mất dữ liệu. Khi đã commit → dữ liệu không mất.		
41	LEVEL 5 – AI SERVICE VALIDATION AI + MCP + Agent validation	ETA model output trong range hợp lý	Model ETA chạy bình thường	distance = 5km	eta > 0 eta < ngưỡng hợp lý (ví dụ < 60 phút) Không trả giá trị vô lý		
42	AI model trả kết quả đúng, ổn định và đáng tin cậy trong hệ	Pricing surge > 1 khi demand cao	demand cao	demand_index = 2	surge > 1 Không vượt quá max (ví dụ 3x)		

43	thống	Fraud score > threshold → flagged	transaction đáng ngờ	hành vi bất thường	fraud_score > threshold Flag = true		
44		Recommendation trả top-3 drivers	gợi ý driver	danh sách driver	Trả top-N (ví dụ 3 driver) Không thiếu / dư		
45		Forecast trả dữ liệu đúng format	dự báo nhu cầu	historical data	Output đúng schema Có timestamp + value		
46		Model version được trả về đúng	nhiều version model	request	Trả đúng model_version Không bị mix version		
47		AI latency < 200ms	service real-time	nhiều request	latency < SLA (ví dụ 200ms) Không timeout		
48		Drift detection trigger	data thay đổi theo thời gian Model học từ dữ liệu cũ → Nhưng thế giới thật đã thay đổi → Model bắt đầu dự đoán sai	input distribution mới	detect drift trigger alert		
49		Model fallback khi lỗi	model lỗi	simulate model crash	fallback sang rule-based không crash system		
50		Input bất thường → model không crash	dữ liệu không chuẩn	outlier (distance = 1000km)	model không crash output vẫn hợp lý hoặc reject		
51	LEVEL 6 – AI AGENT LOGIC Input (context) → Reasoning → Decision (action)	Agent chọn driver gần nhất	nhiều driver available	D1: 5km D2: 2km D3: 3km	Agent chọn D2 Không chọn ngẫu nhiên		
52	AI Agent quyết định:	Agent chọn driver có rating cao hơn	driver có rating khác nhau	D1: distance=2km, rating=4.0 D2: distance=3km, rating=4.9	Agent cân nhắc → có thể chọn D2 Không chỉ dựa vào distance		
53	Chọn driver nào	Agent cân bằng ETA vs price	trade-off	Driver A: ETA=5min, price=50k Driver B: ETA=8min, price=40k	Agent chọn phương án tối ưu (multi-objective)		
54	Giá bao nhiêu ETA là gì Có fraud không	Agent gọi đúng tool (ETA vs Pricing)	cần tính ETA và giá	Đưa ra một luồng nghiệp vụ có liên quan	Gọi đúng service Không gọi dư thừa Không gọi sai thứ tự		
55	Có nên retry / fallback không	Agent xử lý context thiếu dữ liệu	thiếu dữ liệu driver	Đưa ra một luồng nghiệp vụ có liên quan	Agent không crash Fallback hoặc request thêm data		
56	Luồng logic cơ bản: User request	Agent retry khi service lỗi	ETA service fail	Đưa ra một luồng nghiệp vụ có liên quan	Agent retry Không fail ngay		
57	↓ AI Agent lấy context (MCP)	Agent không chọn driver offline	driver list có offline	Đưa ra một luồng nghiệp vụ có liên quan	Loại bỏ driver offline Không assign sai		
58	↓ Gọi các AI services (ETA, Pricing)	Agent log decision đầy đủ	agent hoạt động	Đưa ra một luồng nghiệp vụ có liên quan	Log decision (vì sao chọn driver) Có trace_id		
59	↓ Evaluate	Agent xử lý nhiều request song song	nhiều request cùng lúc	Đưa ra một luồng nghiệp vụ có liên quan	Không conflict Không race condition		
60	↓	Agent fallback rule-based khi AI fail	model crash	Đưa ra một luồng nghiệp vụ có liên quan	Dùng rule-based logic System vẫn chạy		
61	LEVEL 7 – PERFORMANCE & LOAD TEST	1000 requests/second booking	nhiều user đặt xe cùng lúc	1000 request/sec vào /booking	Không crash Response success cao (>95%) Latency ổn định		
62	Hệ thống chịu được bao nhiêu tải và có còn nhanh khi nhiều người dùng không?	ETA service under load	AI service bị gọi nhiều	500 request/sec ETA	ETA vẫn trả đúng Latency < SLA (ví dụ <200ms) Không timeout		

63		Pricing service under spike	surge giờ cao điểm	spike request đột ngột	Không crash Giá vẫn hợp lệ Không trả giá sai		
64		Kafka throughput test	nhiều event (ride_requested)	hàng nghìn event/sec	Không mất event Consumer không bị lag lớn Queue ổn định		
65		DB connection pool exhaustion	quá nhiều connection DB	request concurrent cao	Không vượt max connection Request bị queue hoặc reject Không crash DB		
66		Redis cache hit rate > 90%	cache được sử dụng	nhiều request lặp lại	Cache hit rate > 90% Giảm load DB Response nhanh hơn		
67		API Gateway rate limit	traffic cao	vượt threshold	HTTP 429 Không overload backend Traffic được kiểm soát		
68		P95 latency < 300ms	đo performance Không spike lớn có nghĩa là latency hoặc traffic không tăng đột biến bất thường khi gặp sự tăng đột ngột của traffic (request) trong thời gian rất ngắn	load test P95 = 95% request nhanh hơn giá trị này	P95 latency < 200ms		
69		Load test giờ cao điểm	mô phỏng giờ cao điểm	load tăng dần: Số lượng request (traffic) tăng dần theo thời gian, lưu ý không phải tăng đột ngột như spike	System scale được Không degrade mạnh: Hiệu năng có thể giảm --> điều này là bình thường. Nhưng không được giảm quá nhiều hoặc đột ngột. User vẫn đặt xe được		
70		Auto scaling hoạt động	hệ thống dùng Kubernetes / Swarm	CPU/memory tăng	Pod scale up Load được phân phối Không bottleneck		
71	LEVEL 8 – FAILURE & RESILIENCE Failure: Khi một phần của hệ thống bị lỗi Ví dụ: Service down DB mất kết nối Kafka bị lag Network lỗi Resilience: Khả năng hệ thống vẫn hoạt động hoặc phục hồi nhanh khi có lỗi	Driver service down → fallback	Driver Service không hoạt động	User tạo booking	Không crash toàn hệ thống Booking có thể chuyển sang PENDING thử lại request khi bị lỗi hoặc timeout (Retry) hoặc dùng phương án thay thế khi service fail (Fallback)		
72		Pricing service timeout → retry	Pricing chậm hoặc không phản hồi	create booking	Retry request Hoặc dùng giá mặc định Không fail toàn bộ flow		
73		Kafka down → buffer event	Kafka broker bị lỗi	publish event ride_requested	Event được buffer (retry/outbox) Không mất dữ liệu System không crash		
74		DB failover	DB primary bị down	query DB	Switch sang replica Không downtime lâu Data vẫn consistent		
75		Circuit breaker open	Service lỗi liên tục	gọi Pricing service nhiều lần fail	Circuit breaker mở Ngừng gọi service lỗi Tránh cascade failure		

76		Partial system failure handling	Một phần hệ thống lỗi	booking flow	Phần còn lại vẫn hoạt động Không crash toàn hệ thống		
77		Retry exponential backoff	Service tạm thời lỗi	retry request	Retry sau 1s → 2s → 4s Không spam request Thành công khi service hồi phục		
78		Service mesh routing fail	routing lỗi	request qua mesh: (service mesh) là khi request giữa các microservices không đi trực tiếp, mà đi qua một lớp hạ tầng trung gian gọi là service mesh. Ví dụ: Booking → proxy → proxy → Pricing Việc đi qua các Proxy sẽ giúp cho việc kiểm soát tốt cho các xử lý: routing retry mTLS logging	Fallback route Không mất request		
79		Network partition test	các service không giao tiếp được	simulate network split	System degrade gracefully Không mất dữ liệu Retry khi network hồi phục		
80		Graceful degradation	hệ thống quá tải hoặc lỗi	user request	Tắt bớt feature không quan trọng Vẫn giữ core function (booking) Không crash		

81	LEVEL 9 – SECURITY TEST
	
	kiểm tra hệ thống chống tấn công, bảo vệ dữ liệu và kiểm soát truy cập
		SQL injection attempt	API nhận input từ user (search, login, booking)	{
				"email": "' OR 1=1 --",
				"password": "anything"
				}
					Query không bị bypass
					Không login thành công
					Không lộ dữ liệu DB
					HTTP 400 hoặc 401		

82		XSS input test	User nhập dữ liệu hiển thị trên UI	<script>alert('hack')</script>	Script không được execute Output được escape Không ảnh hưởng UI		

83		JWT tampering	Attacker sửa payload token	{
				"sub": "user_123",
				"role": "USER"
				}
				
				JWT bị thay đổi sub, role
				
				{
				"sub": "admin_001",
				"role": "ADMIN"
				}
					Nếu hệ thống không kiểm tra signature → attacker thành admin
					
					Token decode fail
					HTTP 401
					Không truy cập được API		

84		Unauthorized API access	User không có quyền	gọi API admin	HTTP 403 Forbidden Không trả dữ liệu nhạy cảm		
85		Rate limit attack	Client gửi nhiều request liên tục	Attacker spam API: POST /booking >1000 requests/sec	HTTP 429 Too Many Requests Rate limit hoạt động Không làm sập hệ thống		

86		Replay attack (idempotency)	Attacker gửi lại request cũ	request payment cũ
				
				Request gốc:
				{
				"user_id": "USR123",
				"amount": 50000
				}
				Attacker gửi lại request y hệt:
				{
				"user_id": "USR123",
				"amount": 50000
				}
					Không xử lý lại transaction
					Không bị double charge
					Trả response cũ		

87		Data encryption at rest	Dữ liệu lưu trong DB	Attacker truy cập trực tiếp DB, xem dữ liệu nhạy cảm card_number = 4111111111111111 Cần được mã hóa card_number = "X9f3k2s9...encrypted..."	Data nhạy cảm được mã hóa Không đọc được plaintext Có key management		
88		mTLS communication	Service-to-service communication Nếu dùng TLS thường: Client → Server Client kiểm tra server --> secure Server KHÔNG kiểm tra client --> unsecure Dùng mTLS Client ↔ Server Client kiểm tra server --> secure Server kiểm tra client --> secure Luồng mTLS 1. Client connect server 2. Server gửi certificate 3. Client verify server --> OK 4. Client gửi certificate 5. Server verify client --> OK 6. Secure communication bắt đầu	request không có certificate	Connection bị từ chối Chỉ cho phép mTLS		
89		RBAC enforcement	Role-based access control	user role = DRIVER gọi admin API	HTTP 403 Không truy cập được resource		
90		Sensitive data masking	Trong các hành động Logging data và Response data	API trả về payment info	Không lộ full card number Mask dữ liệu (****1234) Log không chứa thông tin nhạy cảm		

91	LEVEL 10 - ZERO TRUST SECURITY “Never trust, always verify” Không tin bất kỳ request nào—even internal Mỗi request đều phải được xác thực + phân quyền Các thành phần: Authentication, Authorization, mTLS, API Gateway security, Rate limiting, Encryption, Audit logging	Request không có token	Client gọi API nhưng không gửi JWT	GET /booking (không Authorization header)	HTTP 401 Unauthorized Message: "Missing token" Request bị reject ngay tại API Gateway		
92		Token không hợp lệ (tampered)	Token bị sửa payload	JWT đã bị thay đổi signature	HTTP 401 Message: "Invalid token" Không decode được		
93		Token hết hạn	Token đã expired	JWT có exp < current_time	HTTP 401 Message: "Token expired" Không cho truy cập		
94		Service-to-service authentication (mTLS)	Service A gọi Service B	Request không có certificate hợp lệ	Request bị từ chối mTLS handshake fail Không truy cập được service		
95		RBAC – user không có quyền	User thường gọi API admin	Token role = USER Gọi /admin/dashboard	HTTP 403 Forbidden Message: "Access denied"		
96		Least privilege (driver không truy cập user data)	Driver cố truy cập thông tin user khác	driver_id gọi API /users/{user_id}	HTTP 403 Không trả dữ liệu user		
97		API Gateway kiểm tra tất cả request	Request bypass service trực tiếp	gọi trực tiếp service nội bộ (không qua gateway)	Request bị chặn Chỉ cho phép traffic qua API Gateway		
98		Rate limiting chống abuse	Client spam request	>100 requests/second	HTTP 429 Too Many Requests Rate limit được áp dụng Không overload hệ thống		
99		Data encryption in transit	Giao tiếp giữa các service	Request qua HTTP (không TLS)	Request bị từ chối Chỉ cho phép HTTPS / mTLS		
100		Audit logging (security trace)	User truy cập hệ thống	Login + API call	Log đầy đủ: user_id, action, timestamp Có thể trace hành vi Không missing log		
101	LEVEL 11 - DEPLOYMENT Virtual Machine Cluster / Swarm / K8S Mục tiêu: System chạy được trong môi trường thật Không downtime khi update Recover được khi lỗi Các thành phần kiểm thử: Deploy cơ bản, Config & ENV, DB/Kafka connectivity, Scaling, Resilience,	Deploy service thành công (basic)	Deploy một microservice (Booking Service) lên môi trường staging	Docker image hợp lệ Kubernetes manifest đúng	Pod chạy thành công (status = Running) Không crash (CrashLoopBackOff = false) Health check pass (/health = 200)		
102		Health check endpoint	Service đã deploy	GET /health	HTTP 200 Response: "status": "ok" Không timeout		
103		Environment variables đúng	Service cần config (DB, Kafka, Redis)	ENV: DATABASE_URL, KAFKA_BROKER	Service connect được DB/Kafka Không lỗi config missing Logs không có error		
104		Service connect database	DB đã deploy (Postgres)	Service start	Kết nối DB thành công Query test chạy OK Không timeout / connection refused		
105		Service connect Kafka	Kafka cluster hoạt động	Service publish event	Event được gửi thành công Không lỗi broker connection Topic tồn tại		

106	Rollback	Rolling update (zero downtime)	Deploy version mới của service	Update image version	Service không downtime Request vẫn xử lý bình thường Pod cũ bị terminate sau khi pod mới ready		
107		Auto scaling (HPA)	Tải tăng cao	Simulate CPU > threshold	Pod scale từ 2 → N Load được phân phối đều Không quá tải		
108		Service mesh routing	Sử dụng Istio/Linkerd	Request đi qua mesh	Traffic route đúng service mTLS hoạt động Không bị drop request		
109		Config sai → fail fast	ENV sai (DB URL sai)	DATABASE_URL invalid	Service fail ngay khi start Log rõ lỗi Không chạy ở trạng thái “half-broken”		
110		Rollback deployment	Deploy version lỗi	Trigger rollback	Service quay về version trước Hệ thống hoạt động lại bình thường Không mất dữ liệu		
111	LEVEL 12: MONITORING TESTCASES - Giám sát hệ thống (Observability / Monitoring Testing) Mục tiêu: Hệ thống đang hoạt động thế nào trong production Các thành phần: Logging, Metrics, Tracing, Alerting, AI Monitoring, Kafka Monitoring	Logging đầy đủ request	User gọi API /booking	Request booking hợp lệ	Log được ghi lại (request + response) Có request_id / trace_id Không thiếu thông tin quan trọng		
112		Structured logging format	Logging system hoạt động	Trigger API call	Log ở dạng JSON Có fields: timestamp, service_name, level Parse được bởi Elasticsearch		
113		Metrics được expose	Service chạy với monitoring	GET /metrics	HTTP 200 Có metrics: request_count, latency Đọc được bởi Prometheus		
114		Dashboard hiển thị đúng	Metrics đã collect	Xem dashboard	Dashboard hiển thị request rate Latency (p95, p99) đúng Không missing data (Grafana)		
115		Distributed tracing hoạt động	Request đi qua nhiều service	Booking flow (API → AI → Payment)	Trace hiển thị đầy đủ span Có trace_id xuyên suốt Xem được trên Jaeger		
116		Alert khi error rate cao	System bị lỗi nhiều	Simulate error rate > threshold	Alert được trigger Gửi notification (Slack/Email) Không bị delay		
117		Alert khi latency cao	Service chậm	Latency > SLA (ví dụ >500ms)	Alert được trigger Dashboard hiển thị spike Có log liên quan		
118		AI service monitoring	AI model chạy inference	Gọi ETA / Pricing API	Log inference time Track model_version Monitor drift (input distribution)		
119		Kafka monitoring	Event streaming hoạt động	Publish ride_requested	Kafka lag được ghi nhận Consumer offset tracking Không bị backlog lớn		

120		Resource monitoring (CPU, Memory)	System chạy production	Load cao	CPU/memory metrics được ghi nhận Không vượt ngưỡng nguy hiểm Auto scaling trigger nếu cần		
TỔNG PHẦN TRĂM (%)							

Lưu ý	Cá nhân vắng mặt sẽ đạt 0 điểm
	Khi thuyết trình không được sử dụng công cụ hỗ trợ như điện thoại, script giấy….
	Thời lượng báo cáo PROJECT là 120 phút cho cả nhóm
	Nhóm làm sai các yêu cầu về hệ thống sẽ đạt 0 điểm

STT	Mã số sinh viên	Họ và tên	Chữ ký	Điểm PROJECT tính theo phần trăm (%)
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				